Member-only story Featured

Artificial Intelligence Large Language Models LLM AI Technology

Andrej Karpathy's New suggestion- Ask AI output in ASD-STE100 format and not in Text format

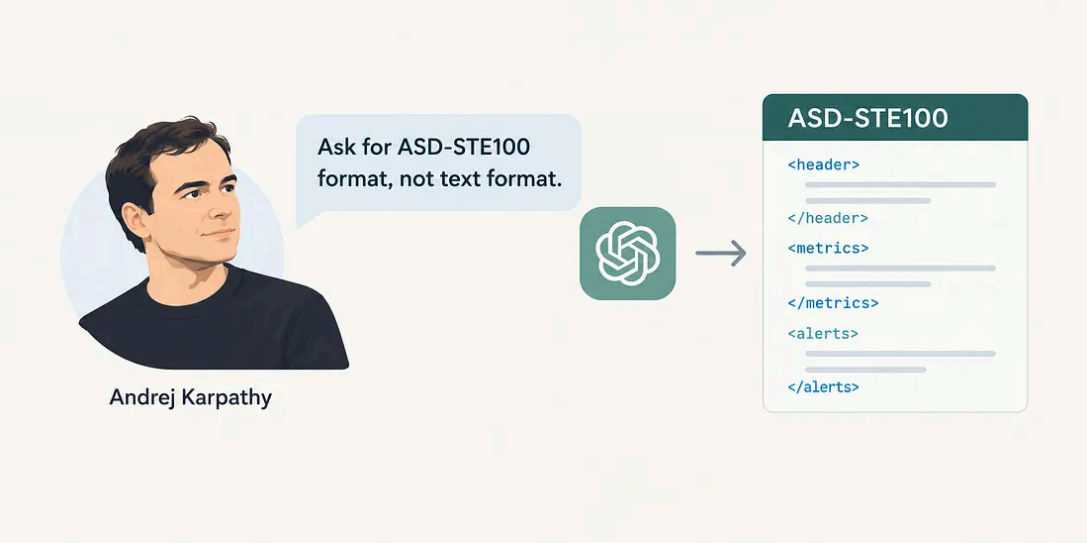
Medium Search Play Games Write

Nikhil Follow 4 min read · Oct 3, 2026

1.3K 16 5

A few days ago, Andrej Karpathy shared a perspective that immediately clicked with how my own daily workflow has been evolving. His premise was simple, as large language models do more and more autonomous work, our primary role as a humans will be towards reviewing, supervising and comprehending what was produced by these models.

The problem in this case is that the raw text output from an AI is often exhausting to read. It tends to ramble, it loves synonyms and it buries key takeaways beneath walls of polite fluff.



Andrej karpathy suggesting to use ASD STE100 format

Karpathy suggested two main fixes to solve this. First borrowing an aviation maintenance standard from 1970's called as **ASD STE100.**

ASD STE100 restricts vocubulary and forces a clear, single action instructions.

Second what he suggested is to moving behind text entirely by asking models for diagrams, interactive HTML pages and throwaway explanatory animations.

I decided to run with both ideas over the past week. the second one (HTML, diagrams was already part of my flow but less frequently). this article is what i learned by sticking to these over the last few days.

The FLUFF Problem

When people say an AI is bad at technical tasks, what they usually mean is that the output feels murky. Models are trained to sound fluent and expansive. In practise, that creates ambiguity.

A model might call a component an engine in paragraph one, a motor in paragraph two and a propulsion unit in paragraph three.

In the aerospace industry, that kind of ambiguity gets people killed.

Back in the late 1970s, European aerospace manufacturers created simplified technical English, known today as ASD STE100. The core guidelines were straightforward.

- Keep procedure sentences under twenty words
- Describe only one action per instruction
- Never switch terms for the sake of variety. If an object is a bolt, call it a bolt forever.

I started adding a simple directive to my technical prompts.

> Explain this using the spirit of ASD STE100. Keep sentence structures direct, limit procedural steps to single actions and avoid unnecessary synonyms.

The explanations from models stopped sounding like a chatty intern trying to pad their word count and started reading like precise workshop manuals. If you review complex codebases, research papers, or architecture proposals, this one constraint saves enormous mental bandwidth.

Ephemeral Artifacts

The second part of Karpathy's observation is where things get genuinely transformative.

For decades, writing custom software was expensive. You only wrote code if thousands of people were going to use it over months or years. But today, the marginal cost of code generation is essentially zero.

That means we can prompt models to produce what Karpathy calls large, custom discardable artifacts.

Here is what that looks like in practise.

Yesterday, i wanted to understand how learning rates affect gradient descent in a specific optimization setup. Previously i would have asked the model to write an explanation. it would have generated four paragraphs describing overshooting and slow convergence.

Instead i gave the below prompt

> *Generate a single, self-contained HTML file with Tailwind CSS and a canvas script. Include an interactive slider for the learning rate, so I can watch a ball roll down an error surface in real time.*

Thirty seconds later, i opened local html files in my browser. I dragged the slider to 0.001 and watched the ball crawl. I moved it to 1.5 and watched the ball violently oscillate off the screen.

The entire explanation took ten seconds of visual interaction to understand completely. Once i understood the mechanish, i closed the tab and deleted the file.

I did not need a reusuable software application. I needed an ephemeral tool whose only purpose was to transfer an insight from the model into my brain as quickly as possible.

New Visual Hierarchy

If your goal is to understand something fast, plain text is actually the lowest bandwidth format you can pick. Here is the hierarchy i now use when prompting models for answers.

1. **Procedural Animations and Video:** Karpathy noted that asking models to write scripts for procedural animations, like the 3Blue1Brown visual style, makes abstract math and algorithms click immediately. Pairing generated visual code with voiceover synthesis turns dense research into a personal lecture.
2. **Interactive HTML Micro Apps:** Whenever numbers or systems are involved, ask for an interactive page with sliders, toggles, or live inputs. Playing with parameters teaches you ten times more than reading static descriptions.
3. **Structured Diagrams:** If an interactive page is overkill, ask for Mermaid diagrams, SVG layouts, or architectural flows. Spatial organization cuts cognitive load instantly.
4. **Constrained Text:** When text is necessary, constrain it. Ask for short sentences, direct verbs, and strict terminology.

Artificial Intelligence ✓ | Large Language Models + | LLM + | AI +
Technology +



Written by Nikhil

1.7K followers · 13 following

Human with a story to tell, a story to preserve and a story still unfolding.

Follow

Learn AI from scratch